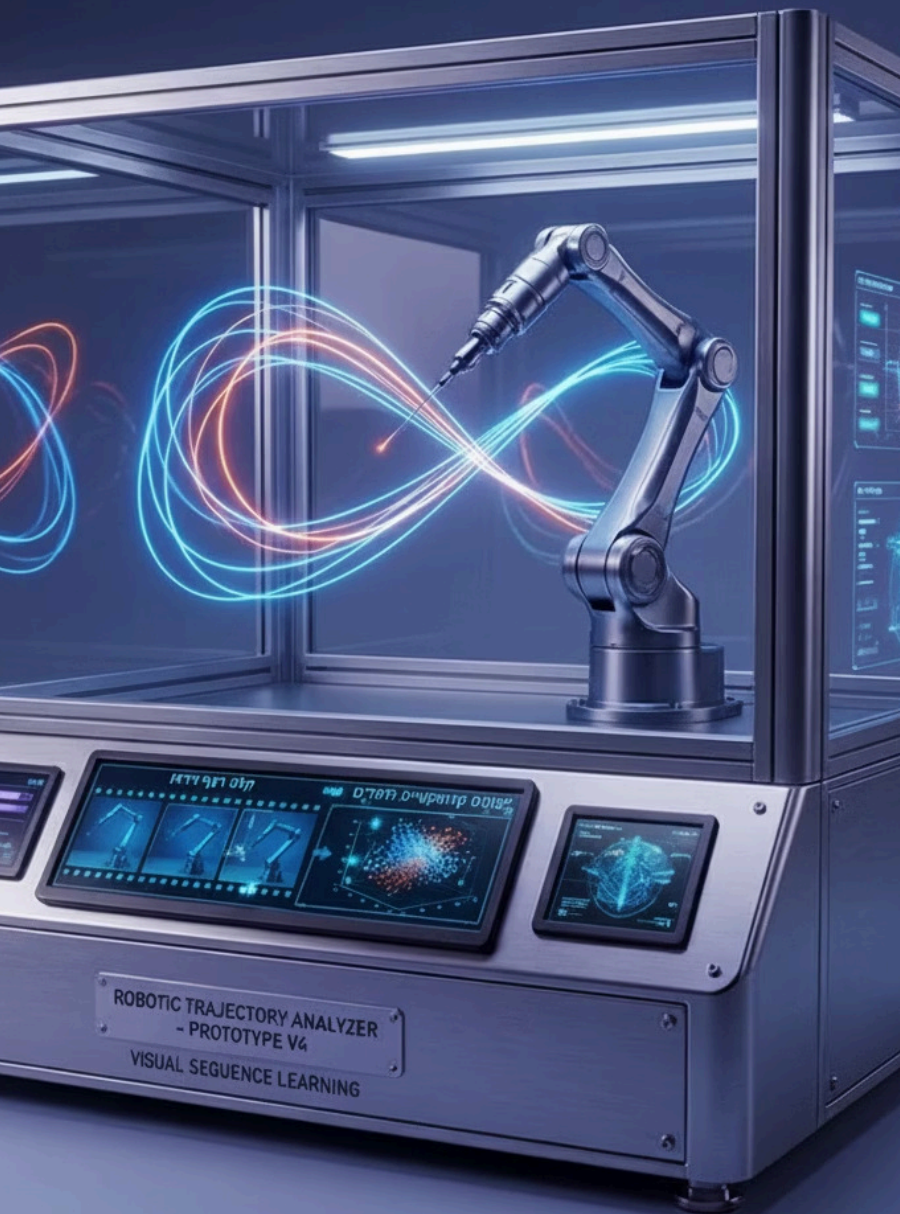




למידת רצפים חזותיים ברובוטיקה באמצעות OpenCLIP ו-Mamba

אוריה כהן

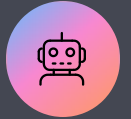
רקע ומוטיבציה: רובוטים לומדים בסביבה דינמית

משימות רובוטיות דורשות להבין רצף של מצבים ופעולות לאורך זמן. כשמשתמשים בתמונות יש הרבה פרטים בכל פריים, ולכן צריך להפוך כל תמונה לייצוג קצר ויעיל לפני שלומדים רצפים.



מידע חזותי = הרבה פרטים

הרובוט "רואה" תמונות כמו מצלמה. בכל תמונה יש הרבה פרטים (תאורה, רקע, זוויות ורעש), ולכן קשה ללמוד ישירות מהתמונה הגולמית.



הכול תלוי בזמן (רצף)

ההחלטה של הרובוט לא נקבעת לפי תמונה אחת, אלא לפי מספר רגעים רצופים: מה קרה קודם ומה משתנה לאורך הזמן.



איזון בין דיוק למהירות

צריך פתרון שגם יהיה מדויק מספיק כדי לזהות פעולה, וגם יעיל כדי להריץ ניסויים בצורה נוחה – בלי עומס חישובי כבד.



רעש ושינויים בסביבה

תאורה, הסתרה חלקית ותנועות קטנות יכולים לשנות את התמונה. לכן חשוב ייצוג יציב שמדגיש את המידע הרלוונטי למשימה.

הבעיה המחקרית: עיבוד רצפים ארוכים ברובוטיקה

עיבוד רצפים ארוכים הוא אתגר מרכזי ברובוטיקה.

יש הרבה פריימים לאורך זמן, ולכן צריך מודל שמסוגל ללמוד תלות בזמן בלי עומס חישובי כבד.

רצפים ארוכים

הבנה ועיבוד של סדרות ארוכות של נתונים חזותיים היא מכשול מרכזי בפיתוח רובוטים חכמים.

מודלים כבדים

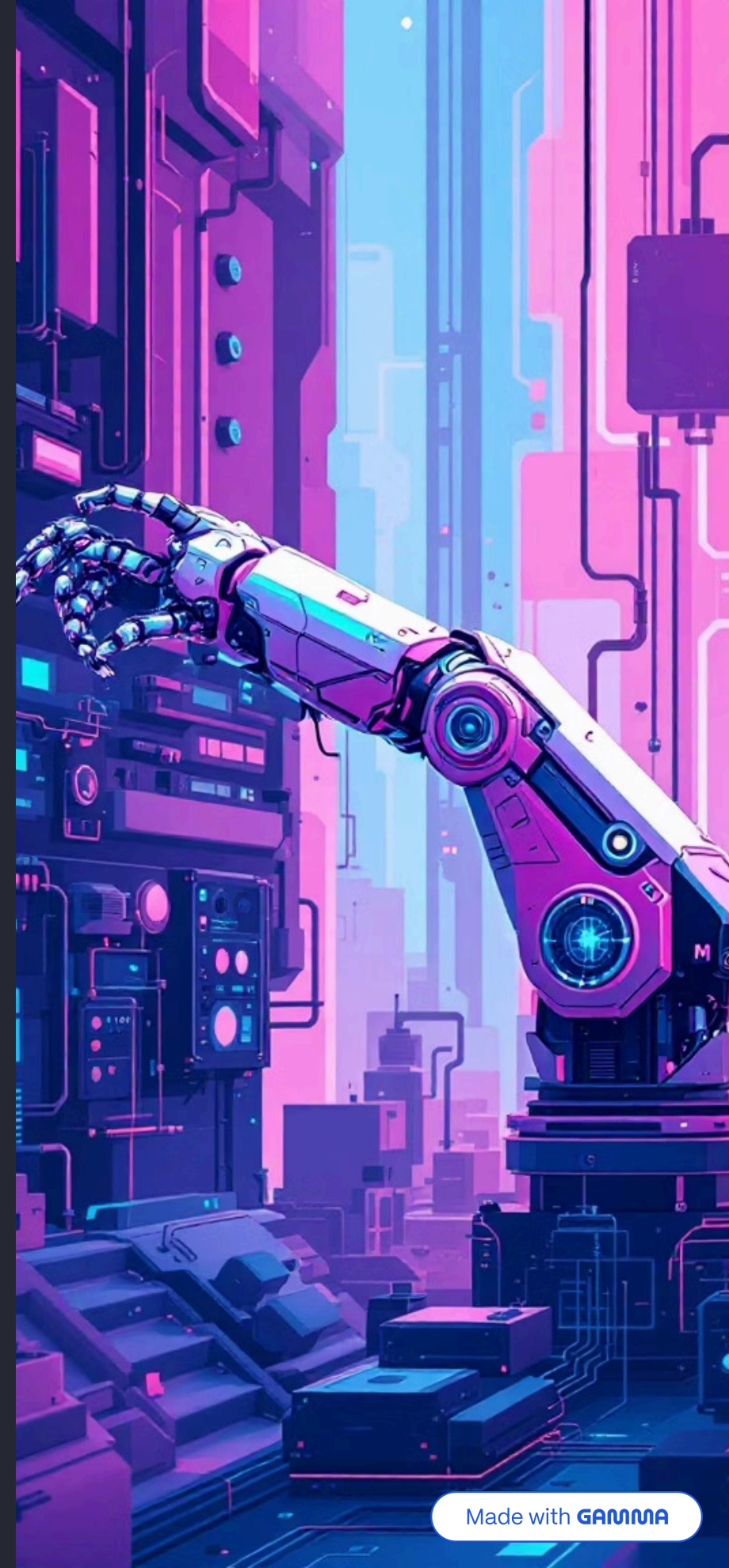
חלק ממודלי הרצפים מציעים דיוק גבוה, אך הם כבדים חישובית ודורשים משאבים רבים – מה שמקשה על שימוש רציף ברובוטיקה.

זמן וזיכרון

רובוטים פועלים תחת אילוצים של זמן תגובה ומשאבים מוגבלים, מה שמחייב פתרונות יעילים יותר.

פתרון יעיל

הדרישה היא לפתרון חלופי שיאפשר ללמוד רצפים באופן יעיל ולאורך זמן, תוך שמירה על ביצועים גבוהים.





ייצוג חזותי (Embedding): תמונה ← וקטור מספרים

במקום להזין למודל את התמונה הגולמית (פיקסלים), אנחנו ממירים אותה ל־Embedding – וקטור קצר שמרכז את המידע החשוב בתמונה.



למה זה חשוב?

זה מאפשר לעבוד עם תמונות בצורה מהירה ויעילה, וללמוד רצפים בלי לעבד פיקסלים כבדים.



מה זה ייצוג חזותי? (Embedding)

זה כמו "סיכום" של תמונה: וקטור קצר של מספרים שמייצג את התכונות החשובות בה.



התוצאה?

במקום תמונה גדולה, יש לנו ייצוג קטן שמאפשר השוואה, למידה וחיזוי לאורך זמן.



איך זה עובד?

מודל מאומן מראש (OpenCLIP) מקבל תמונה ומחזיר ייצוג קבוע (למשל 512 מספרים).

סקירת ספרות

שני מאמרים מרכזיים שמבססים את בחירת המודל והכיוון המחקרי בפרויקט

Mamba as a Motion Encoder for Robotic Imitation Learning (2025)

(Tsuji et al.)

- מה המאמר מציג?
 1. מראה ש-Mamba יכול ללמוד **רצפי תנועה של רובוט** ולחזות תנועה בצורה מדויקת,
 2. במאמר דווח על תנועה יציבה יותר (פחות רעידות) ושיפור ביחס למודלים אחרים בניסויים שלהם.
- למה זה חשוב לפרויקט שלי?

זה נותן חיזוק ש-Mamba מתאים לבעיות של **רצפים רובוטיים בזמן אמת**: צריך יציבות, רציפות, וזיכרון של מה היה קודם.
- איך זה מתחבר בפועל?
 1. ה-**Encoder החזותי הוא OpenCLIP** (תמונה → embedding).
 2. Mamba מקבל רצף של embeddings לאורך זמן ומחזיר חיזוי/החלטה (למשל מצב גריפר / הצלחה).

Mamba – Efficient Sequence Modeling for Long Sequences (2023)

(Gu & Dao et al., arXiv:2312.00752)

- מה המאמר מציג?

מציג את Mamba – מודל שמיועד לעבוד טוב במיוחד על **רצפים ארוכים** (Long Sequences) בצורה יעילה.
- למה זה חשוב לפרויקט שלי?

בפרויקט יש רצף לאורך זמן (פריימים/מצבים). צריך מודל שמסוגל לשמור מידע מהשלבים הקודמים לאורך רצפים ארוכים.
- איך זה מתחבר בפועל?
 1. נשתמש ב-**OpenCLIP** כדי להפוך כל פריים ל-Embedding (וקטור מספרים).
 2. נזין את סדרת ה-embeddings ל-Mamba כדי ללמוד דפוס לאורך זמן (רצף פעולה/מצב/אירוע).

מסקנה: שילוב של Encoder חזותי מוכן מראש (OpenCLIP) עם מודל רצפים יעיל (Mamba) מאפשר ללמוד רצפים ארוכים ברובוטיקה ולבצע חיזוי מדויק לאורך זמן.

השוואת גישות: למה OpenCLIP + Mamba לפרויקט

השוואה לפי יכולת עבודה עם תמונות, למידה לאורך זמן, יעילות ברצפים ארוכים, ושילוב נתוני רובוט.

גישת	קלט חזותי (תמונות)	לומד רצף לאורך זמן	יעיל ברצפים ארוכים	שילוב נתוני רובוט
RNN / LSTM	*✓	✓	X	✓
Transformer	✓	✓	**X	✓
OpenCLIP (בלבד)	✓	X	✓	***✓
OpenCLIP + Mamba (מוצע)	✓	✓	✓	✓

מסקנה: OpenCLIP מספק ייצוג חזותי לכל פריים, ו־Mamba מאחד את הרצף לאורך זמן ומשלב נתוני רובוט.

* ב־RNN/LSTM: כדי לעבוד עם תמונות חייבים להוסיף קודם **Encoder חזותי** (למשל CNN/CLIP) שממיר תמונה ל־embedding.

** Transformer: חזק מאוד ללמידת רצפים, אבל לרוב **כבד יותר** (זמן/זיכרון) ברצפים ארוכים.

*** OpenCLIP בלבד: מפיק **embedding לכל פריים**. אפשר לצרף נתוני רובוט, אבל **אין איחוד רצף לאורך זמן** ללא מודל רצפים נוסף.

הפתרון המוצע: OpenCLIP + Mamba – שילוב מנצח

נשלב שני מודלים:

OpenCLIP הופך כל תמונה ל "מספרים" (embedding), ו-Mamba לומד את הרצף לאורך זמן יחד עם נתוני הרובוט. ככה מקבלים פתרון יעיל שמבין גם תמונות וגם תנועה.



OpenCLIP

ייצוג חזותי לתמונות

- מקבל תמונה ומחזיר וקטור מספרים (embedding).
- כל פריים מקבל ייצוג קבוע (למשל 512 מספרים).
- הייצוג שומר מידע חשוב מהתמונה בצורה קומפקטית.
- **מאומן מראש:** לא צריך לאמן מאפס ← מתחילים מהר.



Mamba

למידת רצף לאורך זמן

- מקבל סדרה של embeddings (פריים אחרי פריים).
- לומד מה משתנה לאורך הזמן (תנועה/שלבים).
- מחזיר חיזוי לרצף (למשל מצב גריפר / הצלחה).
- **יעיל יותר חישובית** מרוב מודלי הרצף הגדולים.



הפרדה ברורה

חלוקת תפקידים

- **שלב 1 – ייצוג חזותי:** ממירים כל תמונה לייצוג מספרי קצר (embedding) בעזרת OpenCLIP.
- **שלב 2 – למידת רצף:** Mamba מקבל את סדרת ה-embeddings ולומד את הקשר לאורך זמן כדי להפיק חיזוי.
- **תוצאה:** חיזוי לאורך זמן + אפשרות לשילוב נתוני רובוט (למשל מיקום/מסלול).



פוטנציאל רובוטי / תוצאה

מה נקבל בסוף?

- חיזוי מצב/פעולה לאורך זמן (למשל: מצב גריפר / הצלחה).
- שילוב תמונות + נתוני רובוט (כמו מיקום/מסלול).
- פתרון קומפקטי שמתאים להרצה ולניסויים.



הדאטה בפרויקט: מקורות מידע ואתגרים

בכל רגע זמן יש לנו: תמונה + מדידת רובוט.
המטרה: ללמוד רצף לאורך זמן (לא רק פריים בודד).

מקורות הנתונים

- רצפי תמונות מהמצלמה/ות בזמן ביצוע המשימה
- מסלול הרובוט (Trajectory): מיקום/כיוון + מצב גריפר לאורך זמן
- סנכרון זמן: התאמה בין תמונה לנתוני רובוט לפי (timestamp)

אתגרי איסוף ועיבוד

- סנכרון: לפעמים התמונה והמדידה לא באותו רגע בדיוק
- רעש בתמונה: תאורה, חסימות (אובייקטים שמסתירים), רקע עמום
- רצפים ארוכים: צריך ללמוד תלות לאורך זמן (לא רק פריים אחד)
- אחדות נתונים: להפוך את כל הדאטה לפורמט אחיד כדי שהמודל ילמד בצורה עקבית

האלגוריתם המוצע (Pipeline): שלב-אחר-שלב

טעינת נתונים

טוענים רצפי תמונות + נתוני תנועה של הרובוט.



הפקת Embeddings

שימוש ב-OpenCLIP כדי להפוך כל תמונה לוקטור מספרים שמייצג "מה רואים".



סנכרון לפי חותמת זמן

משייכים לכל embedding את המצב/הפעולה של הרובוט באותו רגע.



למידת רצף עם Mamba

ה-Mamba לומד את הקשר לאורך זמן בין מה שרואים בתמונות לבין תנועת הרובוט.



בדיקה והערכת ביצועים

בודקים את המודל על כמה משימות ומשווים לגישות אחרות כדי לוודא שהתוצאות נכונות ויציבות.



דרישות: יציבות, יעילות ומודולריות

הפרויקט לא עוסק רק ב"מה עושים", אלא גם ב"איך עושים נכון"

דרישות לא-פונקציונליות

יעילות חישובית

הרצה מהירה יחסית, גם על
רצפים ארוכים, בלי "להיתקע".

יציבות תוצאות

תוצאות עקביות ומובנות על
אותם נתונים/תרחישים.

הפרדת אחריות (Separation of Concerns)

כל רכיב עושה תפקיד אחד
ברור: תמונה ← ייצוג מספרי
(Embedding),
ואז רצף ← החלטה/חיזוי.

הרצה בזמן אמת (אופציונלי)

אפשרות לעיבוד רציף/מהיר
שמתאים ליישומים רובוטיים
בזמן אמת.

דרישות פונקציונליות

טעינת נתונים

קליטה של רצפי תמונות +
נתוני תנועה/מצב של הרובוט.

הפקת ייצוגים חזותיים (Embeddings)

המרת כל תמונה לוקטור
מספרי בעזרת OpenCLIP
(Embedding חזותי).

למידת רצף (Sequence Learning)

למידת הקשר לאורך זמן
בעזרת Mamba על סדרות
ה-embeddings.

חיזוי והצגת תוצאות

הפקת תחזית/סיווג והצגה
ברורה של התוצאות (למשל:
פעולה/מצב).

ממשק משתמש (UI) וסביבת פיתוח

כדי להדגים את הפתרון בצורה ברורה, נבנה ממשק משתמש שמריץ את כל שלבי ה-Pipeline – מטעינת הנתונים ועד הצגת התוצאות.

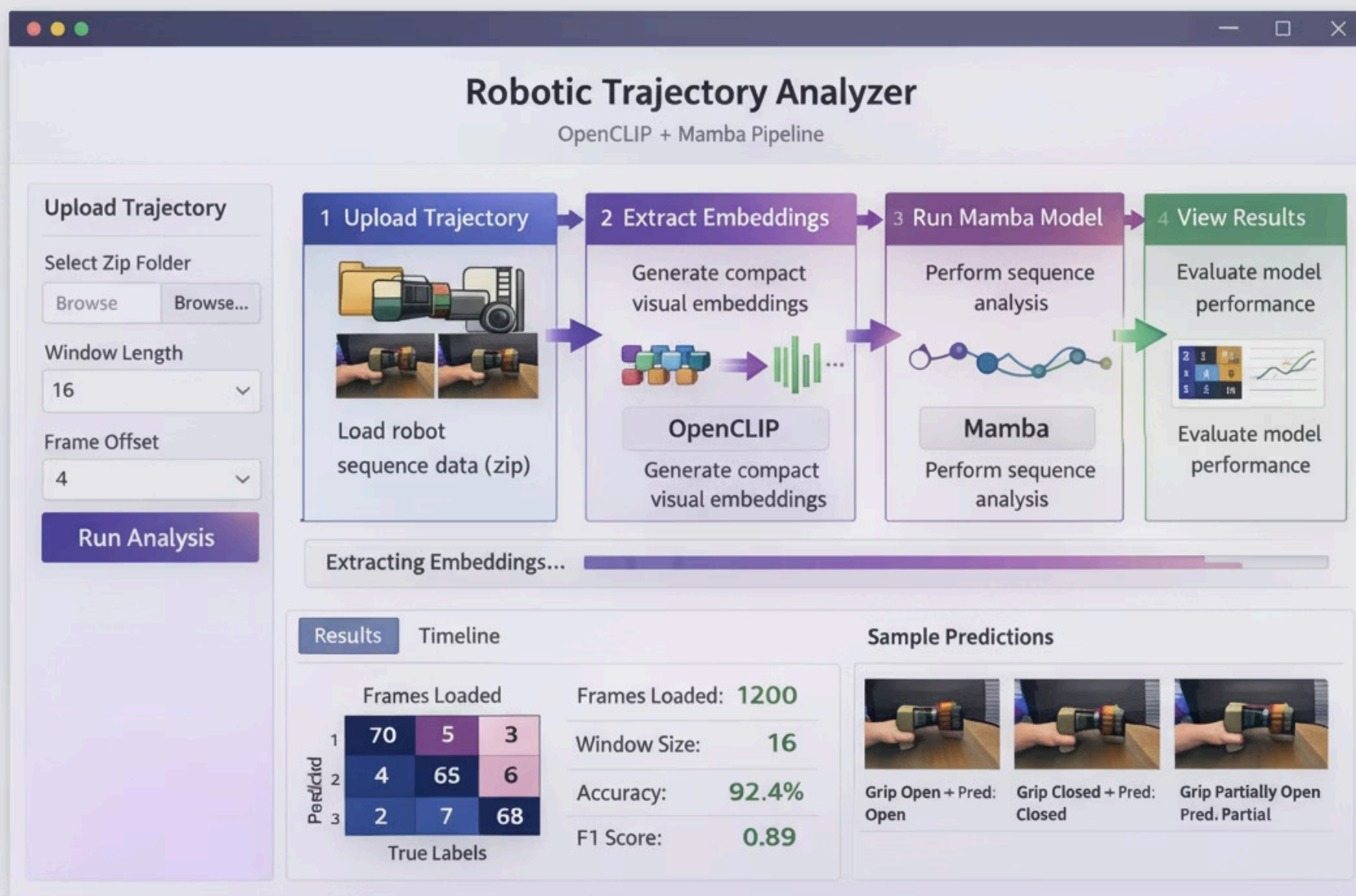
מה הממשק מאפשר (UI)

- **טעינת נתונים:** העלאת דאטה של הרובוט (רצף תמונות + Trajectory).
- **הגדרת פרמטרים (אופציונלי):** בחירת פרמטרים בסיסיים להרצה.
- **הרצה:** הפעלת ה-Pipeline בלחיצה אחת.
- **הצגת תוצאות:** מדדים + גרפים/ציר זמן + דוגמאות.

כלים וסביבת פיתוח (Tools)

- **שפות:** Python
- **ספריות/מודלים:** OpenCLIP ו-Mamba
- **ממשק:** Streamlit
- **פיתוח:** VS Code + סביבת עבודה וירטואלית (venv/conda)

תצוגת ממשק לדוגמה: Robotic Trajectory Analyzer



* פרמטרים להרצה (Window Length / Frame Offset) הם אופציונליים וניתנים לבחירה מחדל.